

Tarea Semana 2: Juego de Adivinanzas

Bucles y Estructuras de Repetición

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido a la Semana 2! Ya dominas los fundamentos de Python y sabes tomar decisiones con condicionales. Ahora es momento de aprender una de las herramientas más poderosas de la programación: **los bucles**.

Los bucles (loops) permiten que tu programa repita acciones múltiples veces sin tener que escribir el mismo código una y otra vez. Es como cuando preparas 10 tazas de café: no escribes 10 veces las instrucciones, simplemente repites el proceso 10 veces.

En esta tarea crearás un juego interactivo de adivinanzas que usa bucles para dar múltiples oportunidades al jugador, validar entradas, generar números aleatorios y crear una experiencia de juego completa y divertida.

2. Objetivos de Aprendizaje

Al completar esta tarea, serás capaz de:

- Usar bucles **while** para repetir código mientras una condición sea verdadera
- Usar bucles **for** para iterar sobre secuencias
- Trabajar con la función **range()** para generar secuencias de números
- Controlar el flujo de bucles con **break** y **continue**
- Validar entradas del usuario usando bucles
- Evitar bucles infinitos mediante condiciones bien diseñadas
- Combinar bucles con condicionales para lógica compleja
- Iterar sobre listas y secuencias
- Usar contadores y acumuladores en bucles

3. Enunciado del Proyecto

3.1. Descripción General

Vas a crear un programa llamado “**Adivinador de Números - Edición Costa Rica**”. Este es un juego interactivo donde:

- El programa genera un número secreto aleatorio
- El jugador tiene un número limitado de intentos para adivinarlo
- El programa da pistas después de cada intento (muy alto, muy bajo, cerca, lejos)
- Se llevan estadísticas del juego
- El jugador puede jugar múltiples rondas
- Hay diferentes niveles de dificultad

El juego debe ser entretenido, dar retroalimentación útil, y demostrar tu dominio de bucles `while`, `for`, `break`, `continue`, y `range()`.

3.2. Modos de Juego

El programa debe ofrecer 3 niveles de dificultad:

1. **Fácil:** Número entre 1 y 50, 10 intentos, pistas detalladas
2. **Medio:** Número entre 1 and 100, 7 intentos, pistas normales
3. **Difícil:** Número entre 1 y 200, 5 intentos, pistas básicas

4. Requisitos Funcionales

4.1. Parte 1: Menú Principal

El programa debe iniciar con un menú que permita:

1. Jugar una nueva partida
2. Ver estadísticas (partidas jugadas, ganadas, perdidas)
3. Ver instrucciones del juego
4. Salir del programa

Implementación requerida:

- Usar un bucle `while` para mantener el menú activo
- Validar que la opción elegida sea válida (1-4)
- Si la opción es inválida, pedir nuevamente (usar `continue`)
- Solo salir cuando el usuario elija la opción correspondiente

4.2. Parte 2: Selección de Dificultad

Cuando el jugador elige jugar:

1. Mostrar los 3 niveles de dificultad con sus características
2. Solicitar que elija un nivel (1-3)
3. Validar la selección con un bucle
4. Configurar el juego según el nivel elegido:
 - Rango de números (máximo)
 - Cantidad de intentos permitidos
 - Nivel de detalle de pistas

4.3. Parte 3: Mecánica del Juego

4.3.1. Generación del Número Secreto

- Usar `import random`
- Generar número aleatorio según el nivel: `random.randint(1, maximo)`
- El número debe mantenerse secreto durante el juego

4.3.2. Bucle Principal del Juego

Implementar con `while` o `for` con `range()`:

1. **Mostrar intento actual:** “Intento 1 de 10”
2. **Solicitar adivinanza del jugador:**
 - Validar que sea un número
 - Validar que esté en el rango válido
 - Si es inválido, no contar como intento y pedir nuevamente
3. **Comparar con el número secreto:**
 - Si adivinó correctamente: ¡Felicitación y terminar con `break`!
 - Si no adivinó: Dar pistas según el nivel
4. **Sistema de pistas:**
 - **Básicas (todos los niveles):** “Muy alto” o “Muy bajo”
 - **Normales (medio y fácil):** “Cerca” (diferencia ≤ 10) o “Lejos”
 - **Detalladas (solo fácil):** “¡Hirviendo!” (diferencia ≤ 5), “Tibio” (6-10), “Frío” (11-20), “Congelado” (21+)
5. **Mostrar intentos restantes:** Después de cada pista
6. **Si se acaban los intentos:** Revelar el número y terminar

4.4. Parte 4: Historial de Intentos

Durante el juego, mantener una lista de todos los números que el jugador ha intentado:

- Usar una lista para guardar cada intento
- Antes de dar pistas, mostrar todos los intentos anteriores
- Usar un bucle `for` para mostrar el historial formateado
- Detectar si el jugador intenta el mismo número dos veces (dar aviso)

4.5. Parte 5: Estadísticas Globales

El programa debe mantener estadísticas entre partidas:

- Total de partidas jugadas
- Partidas ganadas
- Partidas perdidas
- Mejor puntuación (menor número de intentos para ganar)
- Nivel más jugado

Implementación:

- Usar variables que se actualicen después de cada partida
- Mostrar estadísticas cuando el usuario lo solicite del menú
- Usar bucles `for` para mostrar estadísticas formateadas

4.6. Parte 6: Validación de Entradas

Todas las entradas del usuario deben validarse con bucles:

Listing 1: Ejemplo de validación con while

```
1 # Validar que sea un numero
2 while True:
3     entrada = input("Ingresa tu adivinanza: ")
4     if entrada.isdigit():
5         numero = int(entrada)
6         if 1 <= numero <= maximo:
7             break # Entrada valida, salir del bucle
8         else:
9             print(f"Error: Debe estar entre 1 y {maximo}")
10    else:
11        print("Error: Debes ingresar un numero")
```

4.7. Ejemplo de Ejecución Completa

Listing 2: Flujo de juego completo

```
1  =====
2  ADIVINADOR DE NUMEROS - COSTA RICA
3  =====
4
5  --- MENU PRINCIPAL ---
6  1. Jugar nueva partida
7  2. Ver estadísticas
8  3. Instrucciones
9  4. Salir
10
11 Elige una opcion: 2
12
13 --- ESTADISTICAS ---
14 Partidas jugadas: 5
15 Partidas ganadas: 3
16 Partidas perdidas: 2
17 Mejor puntuacion: 4 intentos
18 Nivel mas jugado: Medio
19
20 Presiona ENTER para continuar...
21
22 --- MENU PRINCIPAL ---
23 1. Jugar nueva partida
24 2. Ver estadísticas
25 3. Instrucciones
26 4. Salir
27
28 Elige una opcion: 1
29
30 --- SELECCIONA DIFICULTAD ---
31 1. FACIL - Numero entre 1 y 50 (10 intentos)
32 2. MEDIO - Numero entre 1 y 100 (7 intentos)
33 3. DIFICIL - Numero entre 1 y 200 (5 intentos)
34
35 Elige dificultad: 2
36
37 =====
38 Nivel MEDIO: Adivina el numero entre 1 y 100
39 Tienes 7 intentos. Buena suerte!
40 =====
41
42 Intento 1 de 7
43 Tus intentos anteriores: (ninguno)
44 Ingresa tu adivinanza: 50
45
46 Muy bajo! El numero es mayor que 50.
47 Estas lejos del numero secreto.
48 Intentos restantes: 6
49
50 Intento 2 de 7
```

```
51 Tus intentos anteriores: [50]
52 Ingresa tu adivinanza: 75
53
54 Muy alto! El numero es menor que 75.
55 Estas cerca del numero secreto!
56 Intentos restantes: 5
57
58 Intento 3 de 7
59 Tus intentos anteriores: [50, 75]
60 Ingresa tu adivinanza: 65
61
62 Muy bajo! El numero es mayor que 65.
63 Estas cerca del numero secreto!
64 Intentos restantes: 4
65
66 Intento 4 de 7
67 Tus intentos anteriores: [50, 75, 65]
68 Ingresa tu adivinanza: 70
69
70 Muy bajo! El numero es mayor que 70.
71 Estas muy cerca del numero secreto!
72 Intentos restantes: 3
73
74 Intento 5 de 7
75 Tus intentos anteriores: [50, 75, 65, 70]
76 Ingresa tu adivinanza: 73
77
78 CORRECTO! El numero era 73!
79 Lo adivinaste en 5 intentos!
80
81 --- RESUMEN DE LA PARTIDA ---
82 Numero secreto: 73
83 Intentos usados: 5 de 7
84 Tus intentos: 50, 75, 65, 70, 73
85 Nivel: MEDIO
86 Resultado: GANASTE!
87
88 Quieres jugar otra vez? (s/n): s
89
90 [El juego reinicia...]
```

5. Requisitos Técnicos Detallados

5.1. Uso Obligatorio de Bucles

Tu programa DEBE incluir:

5.1.1. 1. Bucle while para el menú principal

```
1 # Menu que se repite hasta que el usuario elija salir
2 juego_activo = True
3 while juego_activo:
4     # Mostrar menu
5     # Procesar opcion
6     if opcion == 4:
7         juego_activo = False # Salir del bucle
```

5.1.2. 2. Bucle for con range() para intentos

```
1 # Dar un numero limitado de intentos
2 for intento in range(1, max_intentos + 1):
3     print(f"Intento {intento} de {max_intentos}")
4     # Pedir adivinanza
5     # Si adivina correctamente:
6     #     break # Salir del bucle
```

O usar while con contador:

```
1 intento = 1
2 while intento <= max_intentos:
3     print(f"Intento {intento} de {max_intentos}")
4     # Logica del juego
5     intento += 1 # Incrementar contador
```

5.1.3. 3. Bucle while True para validación

```
1 # Validar entrada hasta que sea correcta
2 while True:
3     try:
4         numero = int(input("Ingresa un numero: "))
5         if 1 <= numero <= maximo:
6             break # Entrada valida, salir
7         else:
8             print("Fuera de rango!")
9     except ValueError:
10        print("Debe ser un numero!")
```

5.1.4. 4. Bucle for para iterar sobre listas

```
1 # Mostrar historial de intentos
2 print("Tus intentos anteriores: ", end="")
3 for intento in lista_intentos:
```

```
4     print(intento, end=" ")
5 print()    # Nueva línea
```

5.1.5. 5. Uso de break y continue

```
1 # break: Salir del bucle cuando se adivine
2 if adivinanza == numero_secreto:
3     print("Correcto!")
4     break
5
6 # continue: Saltar iteracion si entrada invalida
7 if not entrada.isdigit():
8     print("Error: Solo numeros")
9     continue # Volver al inicio del bucle
```

5.2. Estructura General del Código

Tu código debe estar organizado así:

```
1 # Encabezado con tu informacion
2 # Nombre, fecha, descripcion
3
4 # Importar libreria random
5 import random
6
7 # Variables globales para estadisticas
8 partidas_jugadas = 0
9 partidas_ganadas = 0
10 partidas_perdidas = 0
11 mejor_puntuacion = float('inf')
12
13 # Bucle principal del programa
14 while True:
15     # Mostrar menu principal
16     # Opcion 1: Jugar
17     # Opcion 2: Ver estadisticas
18     # Opcion 3: Instrucciones
19     # Opcion 4: Salir
```

5.3. Validaciones Requeridas

Debes validar con bucles:

1. **Opción del menú:** Debe ser 1, 2, 3, o 4
2. **Nivel de dificultad:** Debe ser 1, 2, o 3
3. **Adivinanza:** Debe ser un número entero en el rango válido
4. **Jugar otra vez:** Debe ser 's' o 'n' (aceptar mayúsculas/minúsculas)

6. Consejos y Recomendaciones

- **Evita bucles infinitos:** Asegúrate que siempre hay una forma de salir del bucle
- **Usa break sabiamente:** Para salir de bucles cuando se cumpla una condición
- **Usa continue para saltar:** Cuando una iteración no deba completarse
- **Incrementa contadores:** Si usas `while`, no olvides incrementar el contador
- **Prueba diferentes niveles:** Asegúrate que cada dificultad funciona correctamente
- **Prueba casos extremos:**
 - Adivinar en el primer intento
 - Agotar todos los intentos
 - Ingresar el número máximo
 - Ingresar el número mínimo (1)
 - Ingresar valores inválidos
- **Usa variables descriptivas:**
 - `numero_secreto`, no `x`
 - `intento_actual`, no `i`
 - `max_intentos`, no `n`
- **Comenta bucles complejos:** Explica qué hace cada bucle y por qué

7. Entregables

1. Archivo Python llamado `adivinador_numeros.py`
2. El código debe ejecutarse sin errores
3. Debe incluir todos los comentarios requeridos
4. Debe implementar todos los niveles de dificultad
5. Debe mantener estadísticas entre partidas

8. Defensa del Código (60 % de la nota final)

8.1. ¿Qué es la defensa del código?

La defensa del código es una reunión virtual donde presentarás y explicarás tu trabajo. Esta defensa es **obligatoria** y representa el **60 % de la nota final** de esta tarea.

8.2. ¿Cómo funciona?

Durante la defensa deberás:

1. **Compartir tu pantalla** mostrando tu código en el editor.
2. **Ejecutar tu programa** demostrando:
 - El menú principal funcionando
 - Una partida ganada
 - Una partida perdida
 - Las estadísticas actualizándose
 - Validación de entradas inválidas
 - Los 3 niveles de dificultad
3. **Explicar tus bucles:**
 - Por qué elegiste `while` vs `for` en cada caso
 - Cómo funciona tu bucle de validación
 - Qué condición hace que cada bucle termine
 - Cómo usas `break` y `continue`
 - Cómo evitaste bucles infinitos
4. **Responder preguntas como:**
 - “¿Qué pasa si quitas este `break`?”
 - “¿Por qué usaste `range(1, max_intentos + 1)` y no `range(max_intentos)`?”
 - “¿Cómo funciona el sistema de pistas?”
 - “¿Qué hace `continue` en este bucle?”
 - “¿Cómo mantienes las estadísticas entre partidas?”
 - “¿Qué pasaría si el usuario nunca elige salir?”
5. **Mostrar comprensión de:**
 - La diferencia entre `while` y `for`
 - Cuándo usar cada tipo de bucle
 - Cómo funcionan `break` y `continue`
 - Qué es un bucle infinito y cómo evitarlo
 - Cómo iterar sobre listas

8.3. Preguntas Típicas de la Defensa

Prepárate para responder:

- ¿Cuál es la diferencia entre `while` y `for`?
- ¿Por qué elegiste `while True` para la validación?
- ¿Qué hace `range(1, 11)`? ¿Incluye el 11?

- ¿Cómo funciona el **break**? Dame un ejemplo de tu código
- ¿Qué pasaría si olvidas incrementar el contador en un **while**?
- ¿Cómo iteraste sobre la lista de intentos?
- Si cambio **break** por **continue**, ¿qué pasa?
- ¿Cómo garantizas que el menú siempre vuelva a mostrarse?
- ¿Por qué es importante validar antes de procesar?

8.4. Consejos para una Buena Defensa

- **Conoce tus bucles:** Entiende perfectamente cada **while** y **for** que usaste
- **Practica modificaciones:** ¿Qué pasa si cambias una condición?
- **Explica el flujo:** Describe cómo el programa pasa por los bucles
- **Ten ejemplos listos:** Prepara casos de prueba interesantes
- **Entiende break y continue:** Son conceptos clave de esta semana

8.5. Distribución de la Nota

- **Código funcional (40 %):** El programa funciona según los requisitos
- **Defensa del código (60 %):** Tu capacidad de explicar y defender tu trabajo

Importante: Si no asistes a la defensa o no puedes explicar tus bucles, la nota máxima será 40/100.

9. Desafíos Opcionales (Puntos Extra)

1. Modo multijugador (+6 pts):

- 2 jugadores se turnan para adivinar
- Gana quien adivine en menos intentos
- Mantener estadísticas por jugador

2. Pistas avanzadas (+5 pts):

- “El número es par/impar”
- “Es múltiplo de X”
- “Es primo” (si has investigado números primos)

3. Sistema de puntuación (+5 pts):

- Calcular puntos: $(\text{intentos_restantes} \times 10) + \text{bonus_dificultad}$
- Mantener tabla de mejores puntuaciones (top 5)
- Mostrar ranking al finalizar

4. Modo contrarreloj (+6 pts):

- Agregar límite de tiempo usando `time`
- Mostrar tiempo restante
- Terminar si se acaba el tiempo

5. Generador de patrones (+4 pts):

- Además del juego, un modo que genera patrones numéricos con `for`
- Pirámides, secuencias, tablas de multiplicar
- Al menos 3 patrones diferentes

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y puedes explicarla en la defensa.

10. Rúbrica de Evaluación

10.1. Distribución General

- **Código Funcional:** 40 % (40 puntos)
- **Defensa del Código:** 60 % (60 puntos)
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +26 puntos adicionales

10.2. Parte 1: Código Funcional (40 puntos)

10.2.1. 1.1 Implementación de Bucles (18 puntos)

- **Bucle while para menú principal (4 pts):**
 - 4 pts: Menú funcional que se repite hasta salir
 - 3 pts: Funciona pero con pequeños errores
 - 1 pt: Implementación parcial o incorrecta
 - 0 pts: No implementa menú con bucle
- **Bucle para intentos del juego (5 pts):**
 - 5 pts: Usa `for` con `range()` o `while` correctamente
 - 4 pts: Funciona pero lógica mejorable
 - 2 pts: Funciona parcialmente
 - 0 pts: No implementa bucle de intentos
- **Bucles de validación (5 pts):**
 - 5 pts: Valida todas las entradas con bucles apropiados
 - 4 pts: Valida la mayoría de entradas
 - 2 pts: Validación mínima
 - 0 pts: No valida entradas con bucles
- **Uso de `break` y `continue` (4 pts):**
 - 4 pts: Usa ambos correctamente en contextos apropiados
 - 3 pts: Usa uno correctamente
 - 1 pt: Intenta usar pero con errores
 - 0 pts: No usa `break` ni `continue`

10.2.2. 1.2 Funcionalidad del Juego (12 puntos)

- **Generación de número aleatorio (3 pts):**
 - 3 pts: Genera correctamente según nivel de dificultad
 - 2 pts: Genera pero con errores menores
 - 1 pt: Implementación parcial
 - 0 pts: No genera número aleatorio
- **Sistema de pistas (4 pts):**
 - 4 pts: Pistas precisas y diferenciadas por nivel
 - 3 pts: Pistas funcionales pero no diferenciadas
 - 2 pts: Pistas básicas solamente
 - 0 pts: No da pistas
- **Niveles de dificultad (3 pts):**
 - 3 pts: 3 niveles funcionando correctamente
 - 2 pts: 2 niveles funcionando
 - 1 pt: Solo 1 nivel
 - 0 pts: No implementa niveles
- **Detección de victoria/derrota (2 pts):**
 - 2 pts: Detecta ambos casos correctamente y termina apropiadamente
 - 1 pt: Detecta pero con errores
 - 0 pts: No detecta correctamente

10.2.3. 1.3 Características Adicionales (10 puntos)

- **Historial de intentos (4 pts):**
 - 4 pts: Lista completa con iteración usando for
 - 3 pts: Mantiene lista pero presentación mejorable
 - 1 pt: Implementación parcial
 - 0 pts: No mantiene historial
- **Estadísticas globales (4 pts):**
 - 4 pts: Mantiene todas las estadísticas entre partidas
 - 3 pts: Mantiene algunas estadísticas
 - 1 pt: Estadísticas básicas
 - 0 pts: No mantiene estadísticas
- **Opción de jugar múltiples partidas (2 pts):**
 - 2 pts: Permite jugar múltiples veces sin reiniciar programa
 - 1 pt: Funciona con errores
 - 0 pts: Requiere reiniciar programa

10.3. Parte 2: Defensa del Código (60 puntos)

10.3.1. 2.1 Comprensión de Bucles (30 puntos)

■ **Explicación de while vs for (10 pts):**

- 10 pts: Explica perfectamente diferencias y cuándo usar cada uno
- 7 pts: Explica correctamente con pequeñas imprecisiones
- 4 pts: Explicación básica o parcial
- 0 pts: No entiende la diferencia

■ **Explicación de break y continue (8 pts):**

- 8 pts: Explica claramente ambos con ejemplos de su código
- 6 pts: Explica correctamente con alguna duda
- 3 pts: Explicación superficial
- 0 pts: No entiende su funcionamiento

■ **Explicación de range() (6 pts):**

- 6 pts: Explica perfectamente cómo funciona y sus parámetros
- 4 pts: Explica correctamente lo básico
- 2 pts: Explicación confusa
- 0 pts: No entiende range()

■ **Prevención de bucles infinitos (6 pts):**

- 6 pts: Explica cómo garantizó que cada bucle termina
- 4 pts: Comprensión básica del concepto
- 2 pts: Explicación vaga
- 0 pts: No comprende el concepto

10.3.2. 2.2 Demostración y Razonamiento (20 puntos)

■ **Ejecución de diferentes escenarios (8 pts):**

- 8 pts: Demuestra exitosamente múltiples escenarios (ganar, perder, validaciones)
- 6 pts: Demuestra varios escenarios
- 3 pts: Demuestra escenarios básicos
- 0 pts: No puede demostrar diferentes casos

■ **Respuesta a preguntas técnicas (12 pts):**

- 12 pts: Responde correcta y claramente todas las preguntas sobre bucles
- 9 pts: Responde bien la mayoría
- 5 pts: Respuestas parciales
- 2 pts: Dificultad con preguntas básicas
- 0 pts: No puede responder sobre su lógica

10.3.3. 2.3 Calidad del Código (10 puntos)

- **Comentarios explicativos (4 pts):**
 - 4 pts: Comentarios claros explicando bucles complejos
 - 3 pts: Comentarios presentes pero básicos
 - 1 pt: Comentarios mínimos
 - 0 pts: Sin comentarios
- **Nombres de variables (3 pts):**
 - 3 pts: Todos los nombres son descriptivos
 - 2 pts: Mayoría de nombres buenos
 - 1 pt: Nombres poco claros
 - 0 pts: Nombres inapropiados
- **Organización y legibilidad (3 pts):**
 - 3 pts: Código perfectamente organizado y legible
 - 2 pts: Código legible con errores menores
 - 1 pt: Poco organizado
 - 0 pts: Código desorganizado

10.4. Puntos Extra (Hasta +26 puntos)

- **Modo multijugador (+6 pts):** Implementa sistema de turnos entre 2 jugadores
- **Pistas avanzadas (+5 pts):** Pistas matemáticas (par/impar, múltiplos, primos)
- **Sistema de puntuación (+5 pts):** Calcula puntos y mantiene top 5
- **Modo contrarreloj (+6 pts):** Límite de tiempo con módulo `time`
- **Generador de patrones (+4 pts):** Genera 3+ patrones numéricos con bucles

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y el estudiante puede explicarla completamente durante la defensa.

10.5. Penalizaciones

- **-100 % de la defensa:** No asiste sin justificación (nota máxima = 40/100)
- **-50 % de la defensa:** Evidencia de no entender el código
- **-5 pts:** Entrega tardía de 1 día
- **-10 pts:** Entrega tardía de 2-3 días
- **-20 pts:** Entrega tardía de 4+ días
- **-3 pts:** Nombre de archivo incorrecto
- **-5 pts:** Bucle infinito en el código (no termina nunca)
- **-3 pts:** No usa `import random` correctamente

10.6. Nota Mínima Aprobatoria

Para aprobar esta tarea necesitas obtener al menos **60 puntos de 100**.

10.7. Escala de Calificación

- **90-100+ pts:** Excelente - Dominio completo de bucles
- **80-89 pts:** Muy Bueno - Comprensión sólida de estructuras de repetición
- **70-79 pts:** Bueno - Comprensión aceptable, necesita práctica
- **60-69 pts:** Suficiente - Comprensión básica, necesita refuerzo
- **0-59 pts:** Insuficiente - No aprobado

11. Recursos de Apoyo

- **Documentación oficial:** <https://docs.python.org/es/3/tutorial/controlflow.html>
- **Módulo random:** <https://docs.python.org/es/3/library/random.html>
- **Bucles while:** Revisa las notas sobre condiciones de terminación
- **Bucles for:** Practica con `range()` en el intérprete interactivo
- **Break y continue:** Experimenta con ejemplos simples primero

12. Preguntas Frecuentes

P: ¿Cuál es la diferencia entre while y for?

R: `while` repite mientras una condición sea verdadera. `for` repite un número específico de veces o sobre una secuencia.

P: ¿Qué hace `range(1, 11)`?

R: Genera números del 1 al 10 (el límite superior 11 NO se incluye).

P: ¿Cómo evito un bucle infinito?

R: Asegúrate que la condición del `while` eventualmente sea falsa, o que haya un `break` en algún punto.

P: ¿Cuándo uso `break` vs `continue`?

R: `break` sale completamente del bucle. `continue` salta a la siguiente iteración.

P: ¿Cómo genero un número aleatorio?

R: `import random` y luego `random.randint(1, 100)` para un número entre 1 y 100.

P: ¿Puedo usar listas?

R: Sí! Para el historial de intentos es perfecto usar una lista.

P: ¿Es obligatorio hacer los 3 niveles de dificultad?

R: Sí, es un requisito funcional obligatorio.

¡Éxito en tu juego de adivinanzas!

*Los bucles son el corazón de la programación.
Domínalos y podrás automatizar cualquier tarea repetitiva.*